RoboPet — Phase 2, Steps 3 & 4

Module Inventory, rollGacha Cloud Function & Dart GachaService

Goal. Define the module inventory data layer (clearing your ModuleInstance / ModuleRarity analyzer errors), ship the server-authoritative rollGacha Cloud Function with a Remote Config loot table, expose it through a Dart GachaService, and wire the inventory into Riverpod. After this, the economy loop is closed: spend rareCogs -> server rolls -> module lands in Firestore -> streams into Isar via the Phase 2 EconomySyncService.

1. Module data layer (Dart)

These three files clear the static-analysis errors referenced by EconomySyncService._hydrateModules (which uses ModuleInstance, ModuleRarity, and isar.moduleInstances).

1.1 Enums & stat modifiers

lib/data/models/module_enums.dart

```
/// Equipment slots on a robot. Matches the TDD loadout schema.
enum ModuleSlot { wheels, armor, sensor, core, utility }

/// Drop rarities. Order matters for comparisons / sorting.
enum ModuleRarity { common, rare, epic }

/// Which derived combat stat a modifier touches.
enum StatKey { hp, attack, defense, speed, energyRegen }

/// A single additive/multiplicative modifier contributed by a module.
/// resolveStats applies: (base + sum(flat)) * (1 + sum(pct)).
class StatModifier {
  const StatModifier(this.stat, {this.flat = 0, this.pct = 0});
  final StatKey stat;
  final double flat; // absolute add
  final double pct;  // fractional, e.g. 0.15 == +15%
}
```

1.2 Static catalog (ModuleDef)

ModuleDef is static game data -- it is NOT persisted in Isar. It lives in code (and is mirrored in Remote Config for the server). ModuleInstance.defId points at one of these.

lib/data/models/module_def.dart

```
import 'module_enums.dart';

/// Immutable definition of a module type. Static data, compiled in.
class ModuleDef {
  const ModuleDef({
    required this.defId,
    required this.name,
```

```
    required this.slot,
    required this.rarity,
    required this.modifiers,
    this.description = '',
  });

  final String defId;
  final String name;
  final ModuleSlot slot;
  final ModuleRarity rarity;
  final List<StatModifier> modifiers;
  final String description;
}

/// The canonical catalog. The server's Remote Config loot table references
/// these same defIds, so client and Cloud Function never disagree.
const Map<String, ModuleDef> kModuleCatalog = {
  'basic_wheels': ModuleDef(
    defId: 'basic_wheels',
    name: 'Basic Wheels',
    slot: ModuleSlot.wheels,
    rarity: ModuleRarity.common,
    modifiers: [StatModifier(StatKey.speed, flat: 4)],
    description: 'Standard-issue rubber treads.',
  ),
  'mecanum_wheels_mk1': ModuleDef(
    defId: 'mecanum_wheels_mk1',
    name: 'Mecanum Wheels Mk.I',
    slot: ModuleSlot.wheels,
    rarity: ModuleRarity.rare,
    modifiers: [StatModifier(StatKey.speed, flat: 9, pct: 0.05)],
    description: 'Omnidirectional rollers. Strafe like a pro.',
  ),
  'scrap_plating': ModuleDef(
    defId: 'scrap_plating',
    name: 'Scrap Plating',
    slot: ModuleSlot.armor,
    rarity: ModuleRarity.common,
    modifiers: [StatModifier(StatKey.defense, flat: 6)],
    description: 'Welded junk. It mostly holds.',
  ),
  'titan_chassis': ModuleDef(
    defId: 'titan_chassis',
    name: 'Titanium Chassis',
    slot: ModuleSlot.armor,
    rarity: ModuleRarity.epic,
    modifiers: [
      StatModifier(StatKey.defense, flat: 18, pct: 0.10),
      StatModifier(StatKey.hp, flat: 40),
    ],
    description: 'Aerospace-grade frame. Tank mode engaged.',
  ),
  'lidar_array': ModuleDef(
    defId: 'lidar_array',
    name: 'LiDAR Array',
    slot: ModuleSlot.sensor,
    rarity: ModuleRarity.rare,
    modifiers: [StatModifier(StatKey.speed, pct: 0.08), StatModifier(StatKey.attack, flat: 5)],
    description: 'Sees everything. Targets first.',
  ),
  'jetson_nano_core': ModuleDef(
    defId: 'jetson_nano_core',
    name: 'Jetson Nano Logic Board',
    slot: ModuleSlot.core,
    rarity: ModuleRarity.epic,
    modifiers: [
      StatModifier(StatKey.attack, flat: 14, pct: 0.12),
      StatModifier(StatKey.energyRegen, flat: 2),
    ],
```

```
    description: 'On-board inference. Calculates the kill.',
  ),
};
```

1.3 Persisted ownership (ModuleInstance)

lib/data/models/module_instance.dart

```
import 'package:isar/isar.dart';
import 'module_enums.dart';

part 'module_instance.g.dart';

/// A single OWNED module (a roll result). Cloud-authoritative: written by the
/// rollGacha Cloud Function and streamed into Isar by EconomySyncService.
@collection
class ModuleInstance {
  Id id = Isar.autoIncrement;

  /// Firestore document id (users/{uid}/modules/{instanceId}).
  @Index(unique: true, replace: true)
  late String instanceId;

  /// Points at a kModuleCatalog entry / Remote Config def.
  late String defId;

  @Enumerated(EnumType.name)
  ModuleRarity rarity = ModuleRarity.common;

  int level = 1;

  /// Robot instanceId this module is equipped on, or null if in storage.
  @Index()
  String? equippedRobotId;

  late DateTime acquiredAt;

  int schemaVersion = 2;

  /// Convenience: resolve the static def. Returns null if defId is unknown
  /// (e.g. a newer server def the client hasn't shipped yet).
  @ignore
  ModuleDef? get def => kModuleCatalog[defId];
}
```

Errors cleared. ModuleInstance and ModuleRarity now exist, and EconomySyncService._hydrateModules resolves. Re-run codegen: dart run build_runner build --delete-conflicting-outputs

---

2. Remote Config loot table

The server reads the loot table from Firebase Remote Config so you can re-balance without shipping an app update. Create a Remote Config parameter named loot_table_standard_v1 (type JSON) with this default value:

```
{
  "version": 1,
```

```
  "cost": { "currency": "rareCogs", "amount": 1 },
  "pity": { "epicAt": 30 },
  "weights": { "common": 79, "rare": 18, "epic": 3 },
  "pools": {
    "common": ["basic_wheels", "scrap_plating"],
    "rare": ["mecanum_wheels_mk1", "lidar_array"],
    "epic": ["titan_chassis", "jetson_nano_core"]
  }
}
```

Keep the pools defIds in sync with kModuleCatalog. The Cloud Function validates that a drawn defId exists in the pool; unknown client defIds simply render as a generic icon until you ship the matching ModuleDef.

---

3. Cloud Function: rollGacha (TypeScript)

Firebase Functions v2, callable, fully transaction-safe. It deducts currency, applies pity, draws with a crypto-seeded weighted roll, grants the module, bumps economyVersion, and writes an audit record -- all atomically.

3.1 Project bootstrap

```
# From your project root, one time:
firebase init functions      # choose TypeScript, install deps
cd functions
npm install firebase-admin firebase-functions
```

3.2 The function

functions/src/rollGacha.ts

```
import { onCall, HttpsError } from 'firebase-functions/v2/https';
import { getRemoteConfig } from 'firebase-admin/remote-config';
import { getFirestore, FieldValue, Timestamp } from 'firebase-admin/firestore';
import { initializeApp } from 'firebase-admin/app';
import { randomInt, randomUUID } from 'crypto';

initializeApp();
const db = getFirestore();

type Rarity = 'common' | 'rare' | 'epic';
interface LootTable {
  version: number;
  cost: { currency: string; amount: number };
  pity: { epicAt: number };
  weights: Record<Rarity, number>;
  pools: Record<Rarity, string[]>;
}

async function loadLootTable(): Promise<LootTable> {
  const tmpl = await getRemoteConfig().getServerTemplate();
  const cfg = tmpl.evaluate();
  const raw = cfg.getString('loot_table_standard_v1');
  if (!raw) throw new HttpsError('failed-precondition', 'Loot table missing.');
  return JSON.parse(raw) as LootTable;
}
```

```
/** Crypto-seeded weighted rarity draw. */
function drawRarity(weights: Record<Rarity, number>): Rarity {
  const total = weights.common + weights.rare + weights.epic;
  let roll = randomInt(total); // [0, total)
  for (const r of ['common', 'rare', 'epic'] as Rarity[]) {
    if (roll < weights[r]) return r;
    roll -= weights[r];
  }
  return 'common';
}

export const rollGacha = onCall({ region: 'us-central1' }, async (req) => {
  const uid = req.auth?.uid;
  if (!uid) throw new HttpsError('unauthenticated', 'Sign in required.');

  const loot = await loadLootTable();
  const profileRef = db.doc(`users/${uid}/meta/profile`);
  const moduleId = randomUUID();
  const moduleRef = db.doc(`users/${uid}/modules/${moduleId}`);
  const pullRef = db.doc(`users/${uid}/pulls/${moduleId}`);

  return db.runTransaction(async (tx) => {
    const snap = await tx.get(profileRef);
    if (!snap.exists) throw new HttpsError('not-found', 'Profile missing.');
    const data = snap.data()!;

    // 1) Validate & deduct currency.
    const balance = (data.currencies?.[loot.cost.currency] ?? 0) as number;
    if (balance < loot.cost.amount) {
      throw new HttpsError('failed-precondition', 'Insufficient currency.');
    }

    // 2) Pity: force epic at threshold, else weighted draw.
    const pity = (data.pity?.epicCounter ?? 0) as number;
    const forcedEpic = pity + 1 >= loot.pity.epicAt;
    const rarity: Rarity = forcedEpic ? 'epic' : drawRarity(loot.weights);

    // 3) Pick a def from the pool (crypto index).
    const pool = loot.pools[rarity];
    if (!pool || pool.length === 0) {
      throw new HttpsError('internal', `Empty pool: ${rarity}`);
    }
    const defId = pool[randomInt(pool.length)];
    const nextPity = rarity === 'epic' ? 0 : pity + 1;

    // 4) Apply all writes atomically.
    tx.update(profileRef, {
      [`currencies.${loot.cost.currency}`]: FieldValue.increment(-loot.cost.amount),
      'pity.epicCounter': nextPity,
      economyVersion: FieldValue.increment(1),
    });
    tx.set(moduleRef, {
      defId,
      rarity,
      level: 1,
      equippedRobotId: null,
      acquiredAt: Timestamp.now(),
    });
    tx.set(pullRef, {
      defId,
      rarity,
      tableVersion: loot.version,
      cost: loot.cost,
      forcedEpic,
      at: Timestamp.now(),
    });

    return { instanceId: moduleId, defId, rarity, forcedEpic, pity: nextPity };
  });
```

```
});
```

functions/src/index.ts

```
export { rollGacha } from './rollGacha';
```

Deploy:

```
firebase deploy --only functions:rollGacha
```

Anti-cheat guarantees. Currency is read and decremented inside the transaction (no double-spend under concurrent calls). Rarity and def are drawn server-side with crypto.randomInt (never trust a client seed). economyVersion increments on every grant, so the client EconomySyncService adopts the new balance and rejects any stale snapshot. The client cannot write currencies, pity, modules, or pulls (enforced by the Step 1-2 security rules).

---

4. Dart GachaService

Invokes the callable function. Note the client does NOT apply the result locally -- the new module and balance arrive through the existing economy stream, keeping a single source of truth.

lib/data/gacha/gacha_service.dart

```
import 'package:cloud_functions/cloud_functions.dart';
import '../models/module_enums.dart';

class GachaResult {
  GachaResult({
    required this.instanceId,
    required this.defId,
    required this.rarity,
    required this.forcedEpic,
    required this.pity,
  });
  final String instanceId;
  final String defId;
  final ModuleRarity rarity;
  final bool forcedEpic;
  final int pity;
}

class GachaException implements Exception {
  GachaException(this.code, this.message);
  final String code;
  final String message;
  @override
  String toString() => 'GachaException($code): $message';
}

class GachaService {
  GachaService(this._functions);
  final FirebaseFunctions _functions;

  /// Calls rollGacha. Throws GachaException('insufficient'...) on low balance.
  Future<GachaResult> roll() async {
    try {
      final callable = _functions.httpsCallable('rollGacha');
```

```
      final res = await callable.call<Map<String, dynamic>>();
      final d = res.data;
      return GachaResult(
        instanceId: d['instanceId'] as String,
        defId: d['defId'] as String,
        rarity: ModuleRarity.values
            .firstWhere((e) => e.name == d['rarity'] as String),
        forcedEpic: d['forcedEpic'] as bool? ?? false,
        pity: (d['pity'] as num?)?.toInt() ?? 0,
      );
    } on FirebaseFunctionsException catch (e) {
      // 'failed-precondition' => insufficient currency, etc.
      throw GachaException(e.code, e.message ?? 'Gacha roll failed.');
    }
  }
}
```

---

5. Riverpod wiring (inventory + gacha)

Add these to lib/data/providers.dart (alongside the Step 1-2 providers).

```
import 'package:cloud_functions/cloud_functions.dart';

import 'gacha/gacha_service.dart';
import 'models/module_instance.dart';
import 'models/module_enums.dart';

final functionsProvider = Provider<FirebaseFunctions>(
  (ref) => FirebaseFunctions.instanceFor(region: 'us-central1'),
);

final gachaServiceProvider = Provider<GachaService>(
  (ref) => GachaService(ref.watch(functionsProvider)),
);

/// All owned modules, live from Isar (hydrated by EconomySyncService).
final inventoryProvider = StreamProvider<List<ModuleInstance>>((ref) {
  final isar = ref.watch(isarProvider);
  return isar.moduleInstances
      .where()
      .sortByAcquiredAtDesc()
      .watch(fireImmediately: true);
});

/// Modules currently equipped on a given robot, grouped by slot.
final equippedModulesProvider =
    Provider.family<Map<ModuleSlot, ModuleInstance>, String>((ref, robotId) {
  final all = ref.watch(inventoryProvider).valueOrNull ?? const [];
  final map = <ModuleSlot, ModuleInstance>{};
  for (final m in all.where((m) => m.equippedRobotId == robotId)) {
    final slot = m.def?.slot;
    if (slot != null) map[slot] = m;
  }
  return map;
});

/// Imperative roll trigger for the UI. The result module shows up in
/// inventoryProvider automatically via the economy stream.
final rollGachaProvider = FutureProvider.autoDispose<GachaResult>((ref) async {
  return ref.watch(gachaServiceProvider).roll();
});
```

The closed loop: UI calls gachaServiceProvider.roll() -> Cloud Function deducts rareCogs, grants a module, bumps economyVersion -> Firestore pushes the change -> EconomySyncService.watchEconomy() + _hydrateModules write Isar -> currenciesStreamProvider and inventoryProvider update the HUD. No client-side currency math, ever.

---

6. Verification checklist

- [ ] dart run build_runner build --delete-conflicting-outputs succeeds; module_instance.g.dart generated; no analyzer errors for ModuleInstance / ModuleRarity.
- [ ] Remote Config parameter loot_table_standard_v1 published with the JSON above.
- [ ] firebase deploy --only functions:rollGacha succeeds; function shows in the console.
- [ ] With rareCogs >= 1, calling roll deducts 1, creates a modules/{id} doc, and the new module appears in the inventory list within a second.
- [ ] With rareCogs == 0, the call throws failed-precondition and balance is unchanged.
- [ ] After 29 non-epic pulls, the 30th is guaranteed epic and pity.epicCounter resets to 0.

Phase 2 complete after this. Next up is Phase 3 -- Combat (auto-battler resolution using resolveStats over equipped modules, async ghost PvP snapshots).